



M0 基本概念

高秀娟

April of 2022

課程大綱

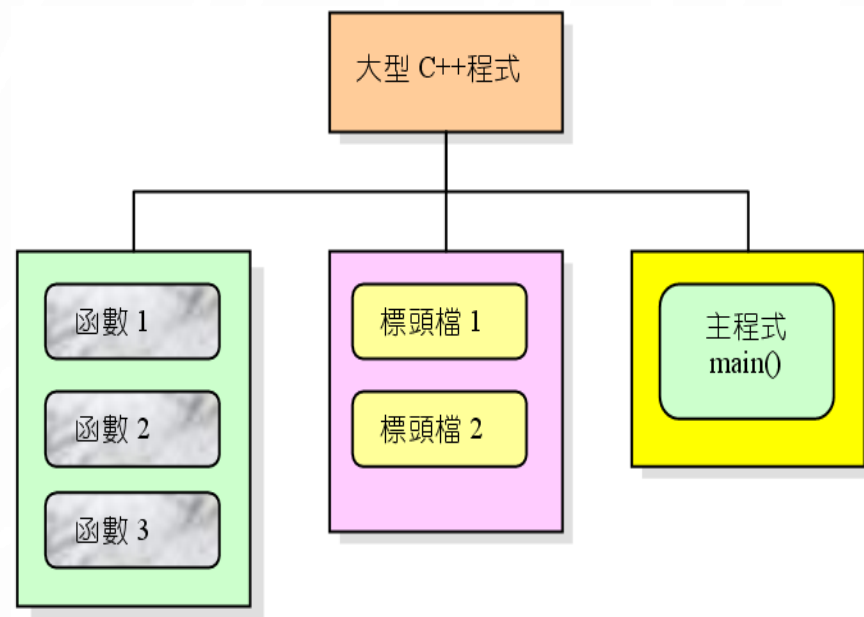
- 程式的模組化
- 結構的資料型態
- 常用的組合邏輯電路
- STM32F072功能介紹與實作
- About ARM



程式的模組化

程式的模組化

- 當我們累積了許多函式之後，可以依照函式的功能性或相關性來進行歸類與打包，將幾個功能相關的函式放在同一個檔案中，建立檔名，稱為一個模組(Module)。對於程式開發者而言，直接使用現成的模組可以加快程式開發速度，讓程式更具可讀性、可擴充性，而且容易維護。
- 欲使用模組時，只要以 `#include` 敘述匯入**標頭檔**，就可以取用模組內的函式。標頭檔中包含模組中所定義的函式名稱和global變數，如下圖所示。標頭檔可以被多次引入，因为在編譯時，編譯器會有防止重複引入的機制。
- 插入系統內建的標頭檔：`#include`會到系統所設定的目錄找尋被含括的檔案。Ex: `#include <stdio.h>`
- 自訂的標頭檔：前置處理器則會在目前的目錄中尋找該標頭檔案，並找到相關的函式。Ex: `#include "area.h"`



C 的 extern 用法

- 變數使用前要先宣告，C 的 extern 關鍵字，用來表示此變數已經在別處定義，告知編譯器到別的地方找尋此變數的定義。



結構資料型態

結構的宣告

- 結構可以同時存放不同型態的資料於同一個結構體
- 結構的定義及宣告格式如下

```
struct 結構名稱 {  
    資料型態 欄位名稱 1;  
    資料型態 欄位名稱 2;  
    ...  
    資料型態 欄位名稱 n;  
};  
struct 結構名稱 變數 1, 變數 2, ..., 變數 m;
```

→ 不需要加分號

→ 記得要加分號

```
struct 結構名稱 {  
    資料型態 欄位名稱 1;  
    資料型態 欄位名稱 2;  
    ...  
    資料型態 欄位名稱 n;  
} 變數 1, 變數 2, ..., 變數 m;
```

→ 不需要加分號

```
struct mydata // 定義結構 mydata  
{  
    string name; // 各欄位的內容  
    string id;  
    int math;  
    int eng;  
};
```

```
struct mydata  
{  
    string name;  
    string id;  
    int math;  
    int eng;  
} student;
```

結構變數的使用及初值的設定

- 結構變數的結構內的成員可以利用小數點 (.) 來存取

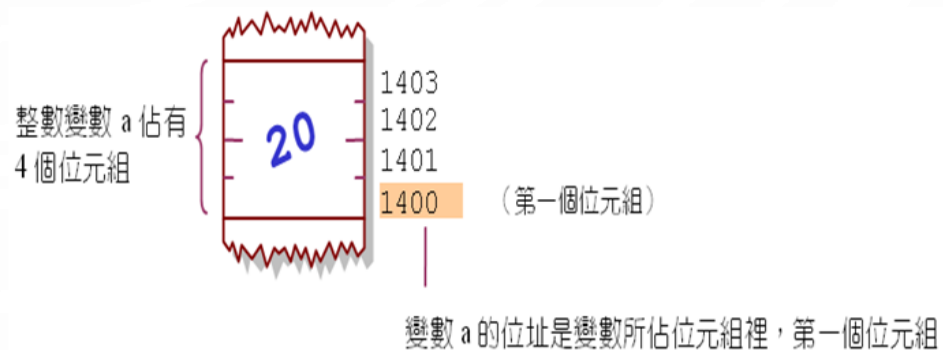
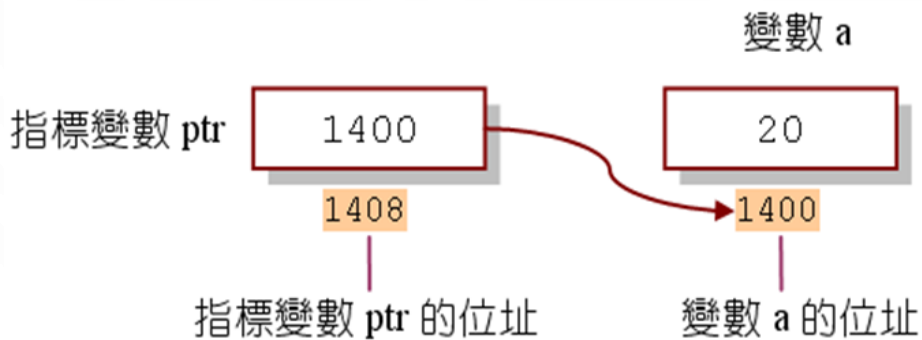
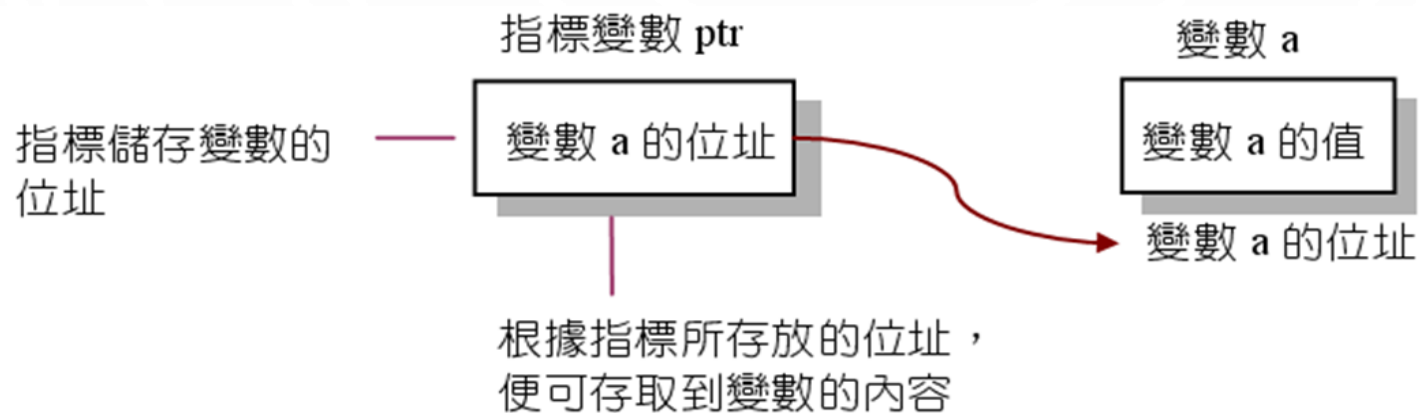
結構變數名稱 . 欄位名稱

```
struct mygood          // 定義結構 mygood
{
    string good;         // 貨品名稱
    int cost;            // 貨品成本
};
struct mygood first={"cracker",32};
```

```
struct mygood          // 定義結構 mygood
{
    string good;         // 貨品名稱
    int cost;            // 貨品成本
} first={"cracker",32}; // 同時宣告變數 first, 並設定初值
```

什麼是指標

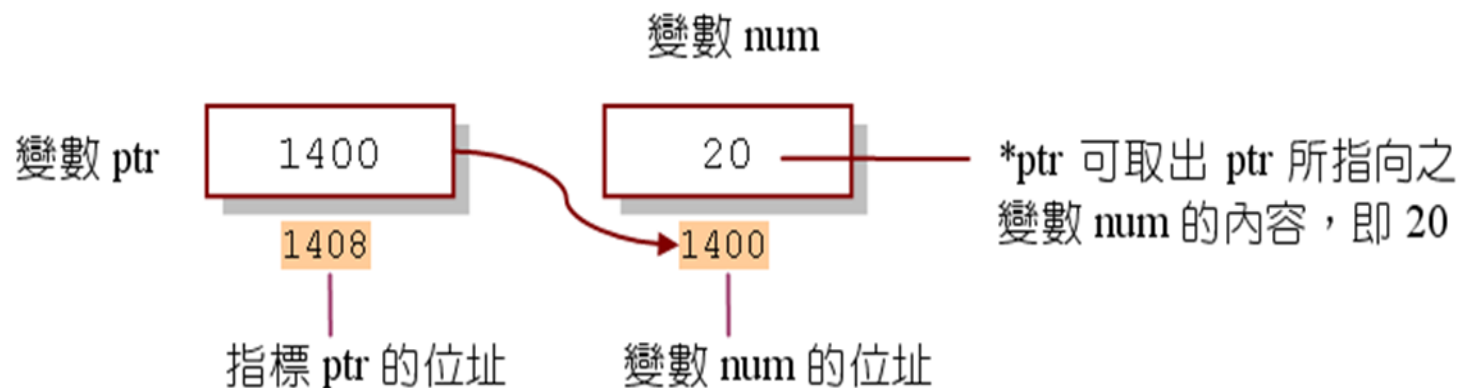
- 指標（**pointer**）是用來存放變數在記憶體中的位址。為什麼要用指標？更有效率。較複雜的資料結構，需要指標才能將資料鏈結在一起。許多函數必須利用指標來傳達記憶體的位址。
- 如果指標**ptr**存放變數**a**的位址，則下面是指標**ptr**指向變數**a**的示意圖



指標變數的宣告

- 把指標ptr指向整數變數num
- 位址運算子「&」可用來取得變數的位址
- 依址取值運算子「*」可取得指標所指向變數的內容. 舉例來說，*ptr可取得num的值.

```
int num=20;  
int *ptr=&num;
```



結構與指標

- 宣告結構型態的指標與宣告一般資料型態的指標一樣,只是存取指標成員資料時,使用指標運算符號 ->

```
struct mydata    // 定義結構 mydata
{
    string name;  // 各欄位的內容
    string id;
    int math;
    int eng;
};
```

```
struct mydata student;
struct mydata *data_ptr = &student;
```

```
student.name = "Julia";
student.id = "30";
```

```
data_ptr->name = "Julia";
data_ptr->id = "30";
```

使用自訂的型態 TYPEDEF

- **typedef**是**type definition**的縮寫，就是定義型態之意。利用**typedef**可以將已經有的資料型態重新定義其識別名稱,它可以讓我們定義屬於自己的資料型態,讓程式的宣告變得較為清楚.
- **typedef**的使用格式如下所示.

```
typedef 資料型態 識別字;
```

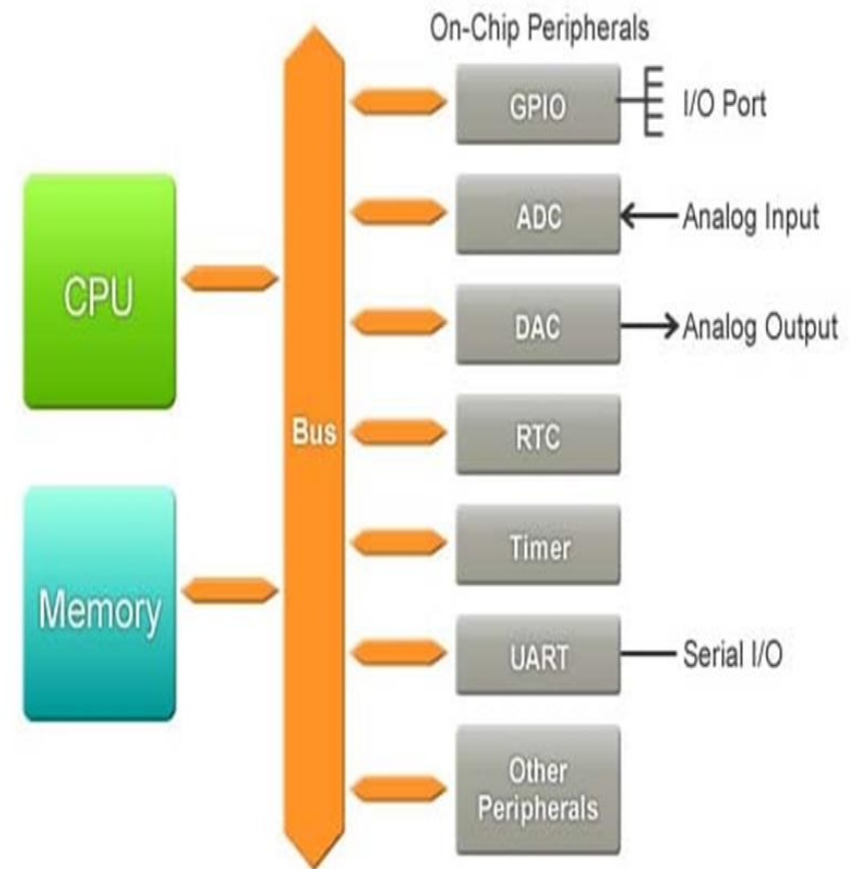
```
typedef int clock;    // 定義 clock 為整數型態  
clock hour, second;  // 宣告 hour, second 為 clock 型態
```

```
Typedef struct {  
    string name;  
    string id;  
    int math;  
    int eng;  
} mydata;
```

```
mydata student;  
mydata *data_ptr = &student;  
  
data_ptr->name = "Julia";  
data_ptr->id = "30";
```

輸入/輸出暫存器

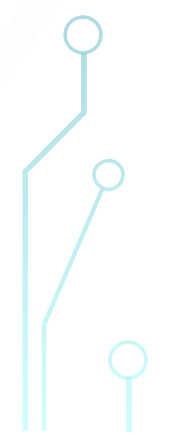
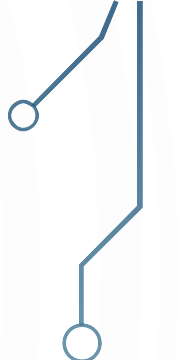
- 一般CPU與外界的溝通是透過輸入/輸出port，I/O port 通常由資料暫存器組成。CPU 可以讀取input port，取得外部訊號的狀態，或是把資料寫入Output port，用以控制外部元件。
- I/O port一般會被分配到與記憶體不同的位址，所以CPU可以用與存取記憶體的方式一樣，去存取I/O port的資料。



Define the peripheral register set as a data structure

- To simplify the code, we can define the peripheral register set as a data structure and define the specific peripheral name as a memory pointer to this data structure.
- For example: General Purpose I/O control register

```
typedef struct
{
    __IO uint32_t MODER;           /*!< GPIO port mode register,           Address offset: 0x00 */
    __IO uint32_t OTYPER;          /*!< GPIO port output type register,      Address offset: 0x04 */
    __IO uint32_t OSPEEDR;         /*!< GPIO port output speed register,     Address offset: 0x08 */
    __IO uint32_t PUPDR;           /*!< GPIO port pull-up/pull-down register, Address offset: 0x0C */
    __IO uint32_t IDR;             /*!< GPIO port input data register,       Address offset: 0x10 */
    __IO uint32_t ODR;             /*!< GPIO port output data register,      Address offset: 0x14 */
    __IO uint32_t BSRR;            /*!< GPIO port bit set/reset register,   Address offset: 0x1A */
    __IO uint32_t LCKR;            /*!< GPIO port configuration lock register, Address offset: 0x1C */
    __IO uint32_t AFR[2];          /*!< GPIO alternate function low register, Address offset: 0x20-0x24 */
    __IO uint32_t BRR;            /*!< GPIO bit reset register,           Address offset: 0x28 */
} GPIO_TypeDef;
```



Define the peripheral register set as a data structure

```
#define FLASH_BASE      ((uint32_t)0x08000000U)          /*!< FLASH base address in the alias region */
#define FLASH_BANK1_END ((uint32_t)0x0801FFFFU) /*!< FLASH END address of bank1 */
#define SRAM_BASE       ((uint32_t)0x20000000U)          /*!< SRAM base address in the alias region */
#define PERIPH_BASE     ((uint32_t)0x40000000U)          /*!< Peripheral base address in the alias region */

/*!< Peripheral memory map */
#define APBPERIPH_BASE  PERIPH_BASE
#define AHBPERIPH_BASE  (PERIPH_BASE + 0x00020000)
#define AHB2PERIPH_BASE (PERIPH_BASE + 0x08000000)
```

■ Define a peripheral register as a pointer

- `// GPIOA_BASE=0x4800 0000`
- `#define GPIOA_BASE (AHB2PERIPH_BASE + 0x00000000)`
- `//指到GPIOA_BASE位址的指標`
- `#define GPIOA ( (GPIO_TypeDef *) GPIOA_BASE )`

■ Ex: 設定GPIOA暫存器的值

- `GPIOA->MODER &= ~ (0x3ul << 2*5);` `// set GPIOA pin 5 as output pin`
- `GPIOA->MODER |= (0x1ul << 2*5);`

STMF072 週邊硬體存取方式

- CPU存取週邊硬體的方式,如果與CPU存取記憶體的方式一樣,都是透過資料和位址匯流排(address and data bus)去存取資料,就稱為memory-mapped I/O. 記憶體和週邊硬體(I/O port)會被分配到不同的位址. 使用這樣的設計,可以使用跟記憶體存取相同的指令去存取週邊硬體(I/O port).
- APB address: 0x4000 0000 – 0x4000 8000 , 0x4001 0000 - 0x4001 8000
- AHB1 address: 0x4002 0000 – 0x4002 43FF
- AHB2 address: 0x4800 0000 – 0x4800 17FF

```
#define FLASH_BASE          ((uint32_t)0x08000000U)          /*!< FLASH base address in the alias region */
#define FLASH_BANK1_END     ((uint32_t)0x0801FFFFU) /*!< FLASH END address of bank1 */
#define SRAM_BASE           ((uint32_t)0x20000000U)          /*!< SRAM base address in the alias region */
#define PERIPH_BASE         ((uint32_t)0x40000000U)          /*!< Peripheral base address in the alias region */

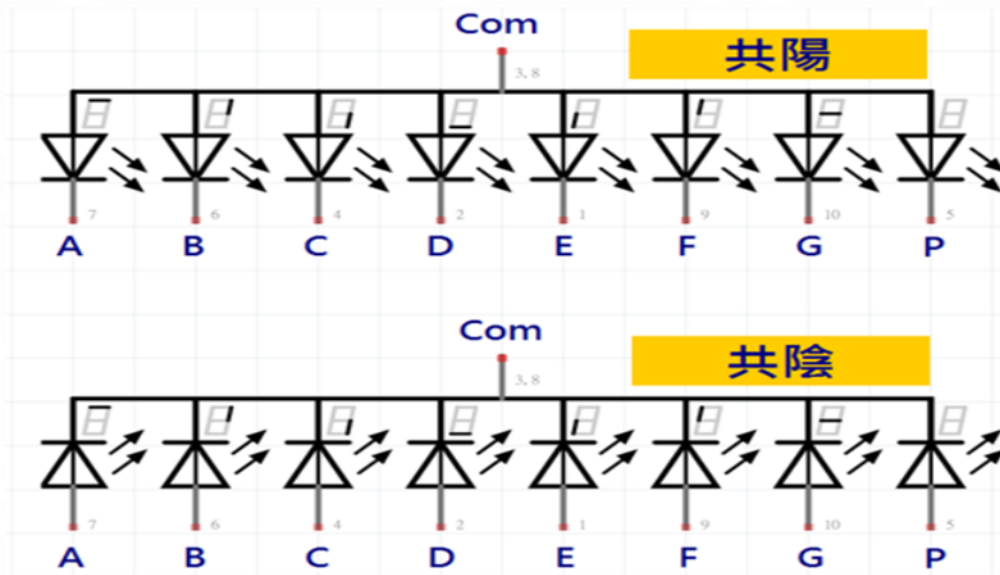
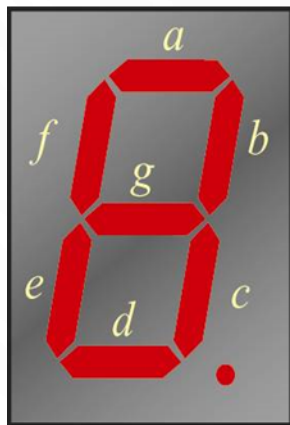
/*!< Peripheral memory map */
#define APBPERIPH_BASE      PERIPH_BASE
#define AHBPERIPH_BASE      (PERIPH_BASE + 0x00020000)
#define AHB2PERIPH_BASE     (PERIPH_BASE + 0x08000000)
```



MCU 中常用的組合邏輯電路

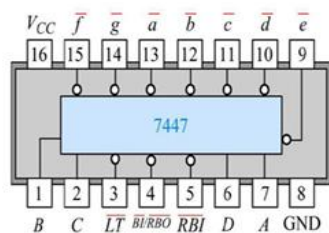
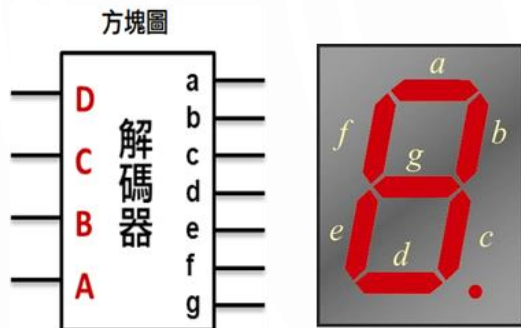
7段顯示器特性

- 7段顯示器是由7個LED顯示片段a,b,c,d,e,f,g所組成, dp 是小數點.利用7段顯示器可以顯示0 - 9的數字.
- 7段顯示器分為共陽型和共陰型
 - 共陽型: V_{cc} 為共同點,輸入腳位A,B,C,D,E,F,G為低電位時,LED會發亮
 - 共陰型: GND 為共同點,輸入腳位A,B,C,D,E,F,G為高電位時,LED會發亮

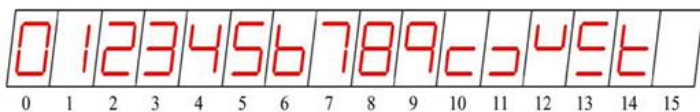


利用IC 7447 BCD解碼器控制7段顯示器

共陽型的7段顯示器真值表



(a) 7447 接腳圖

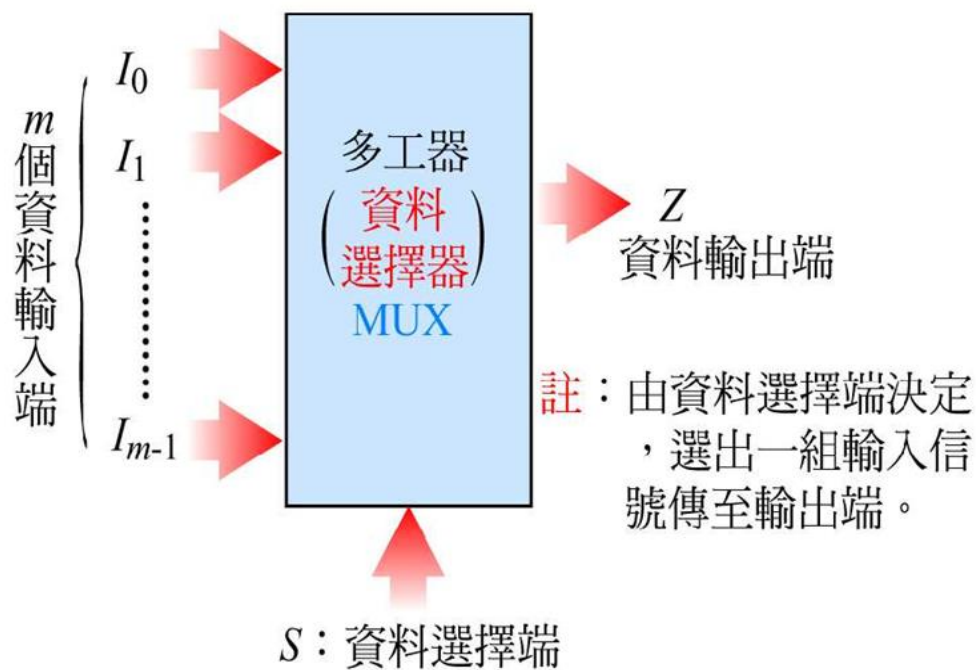


(b) 顯示字型

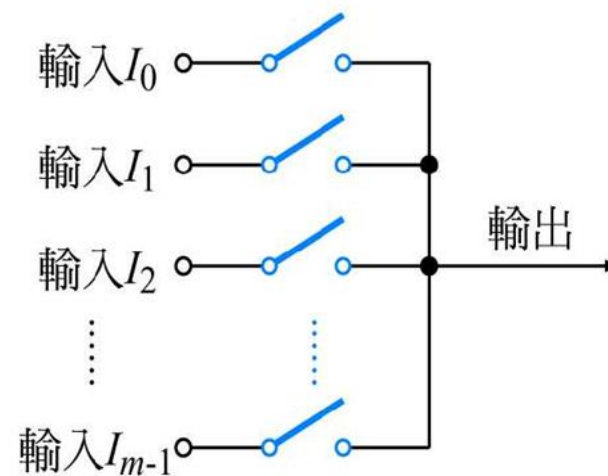
D	C	B	A	a	b	c	d	e	f	g	數字
0	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	1	0	0	1	1	1	1	1
0	0	1	0	0	0	1	0	0	1	0	2
0	0	1	1	0	0	0	0	1	1	0	3
0	1	0	0	1	0	0	1	1	0	0	4
0	1	0	1	0	1	0	0	1	0	0	5
0	1	1	0	0	1	0	0	0	0	0	6
0	1	1	1	0	0	0	1	1	1	1	7
1	0	0	0	0	0	0	0	0	0	0	8
1	0	0	1	0	0	0	0	1	0	0	9

多工器(multiplexer, mux)

- 多工器或稱為資料選擇器(data selector),它主要的功能是從許多條資料輸入線,選擇其中一條輸入資料,送至單一輸出線上.至於選擇那一條輸入,由選擇訊號(S)決定.



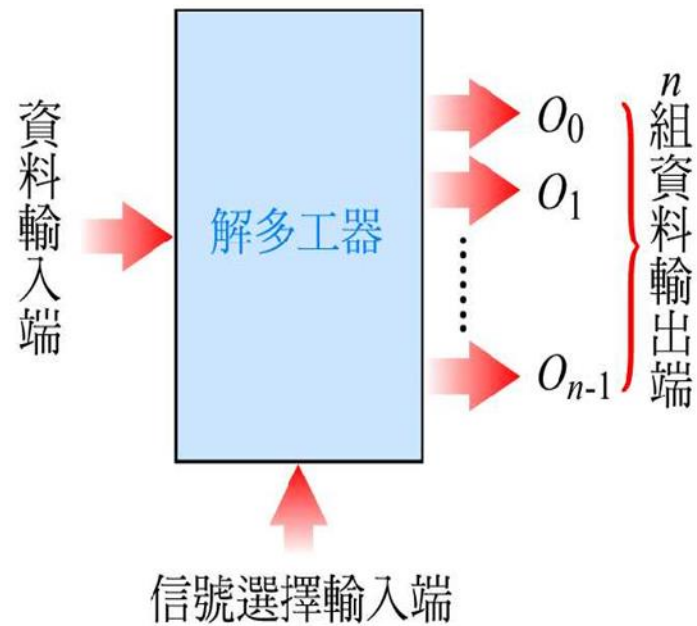
(a)多工器示意圖



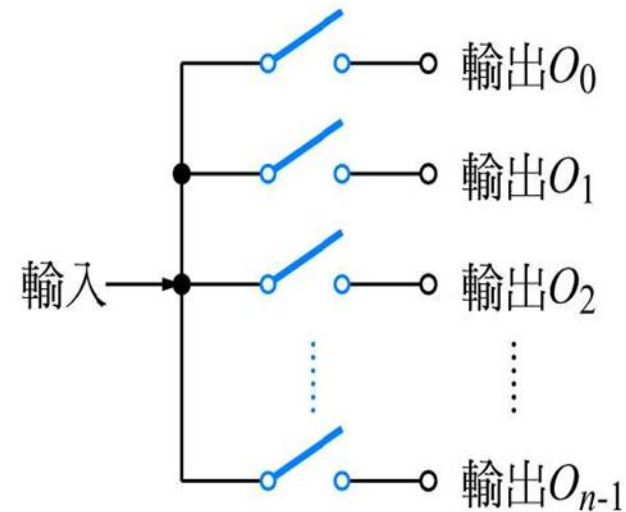
(b)多工器結構簡示圖

解多工器(de-multiplexer, demux)

- 解多工器或稱為資料分配器(data distributor),它的動作和多工器恰好相反,它只有一條輸入資料線,但有多條輸出線.選擇其中一條輸出線,將資料送至被選擇的輸出線上.



(a)解多工器示意圖



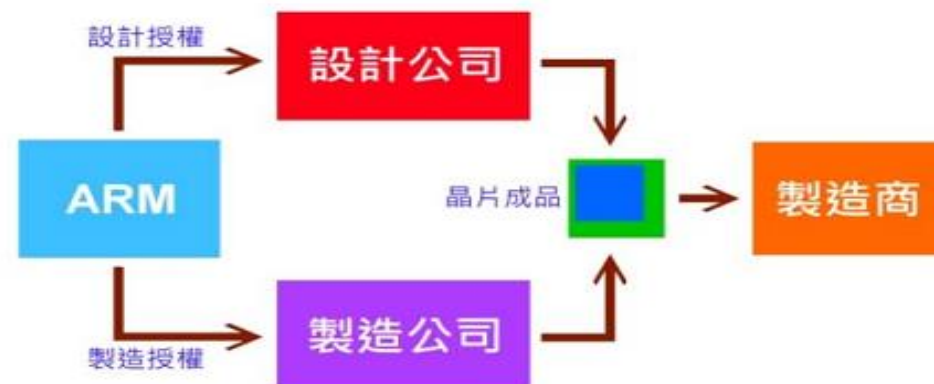
(b)解多工器結構簡示圖

STM32F072 功能介紹與實作

- GPIO : control led、relay
- NVIC (中斷) : touch pad
- Timer / PWM : motion sensor 、buzzer 、motor control 、環境溫濕度偵測
- UART : BT 、car control 、Lora
- ADC : range detection 、土壤濕度偵測

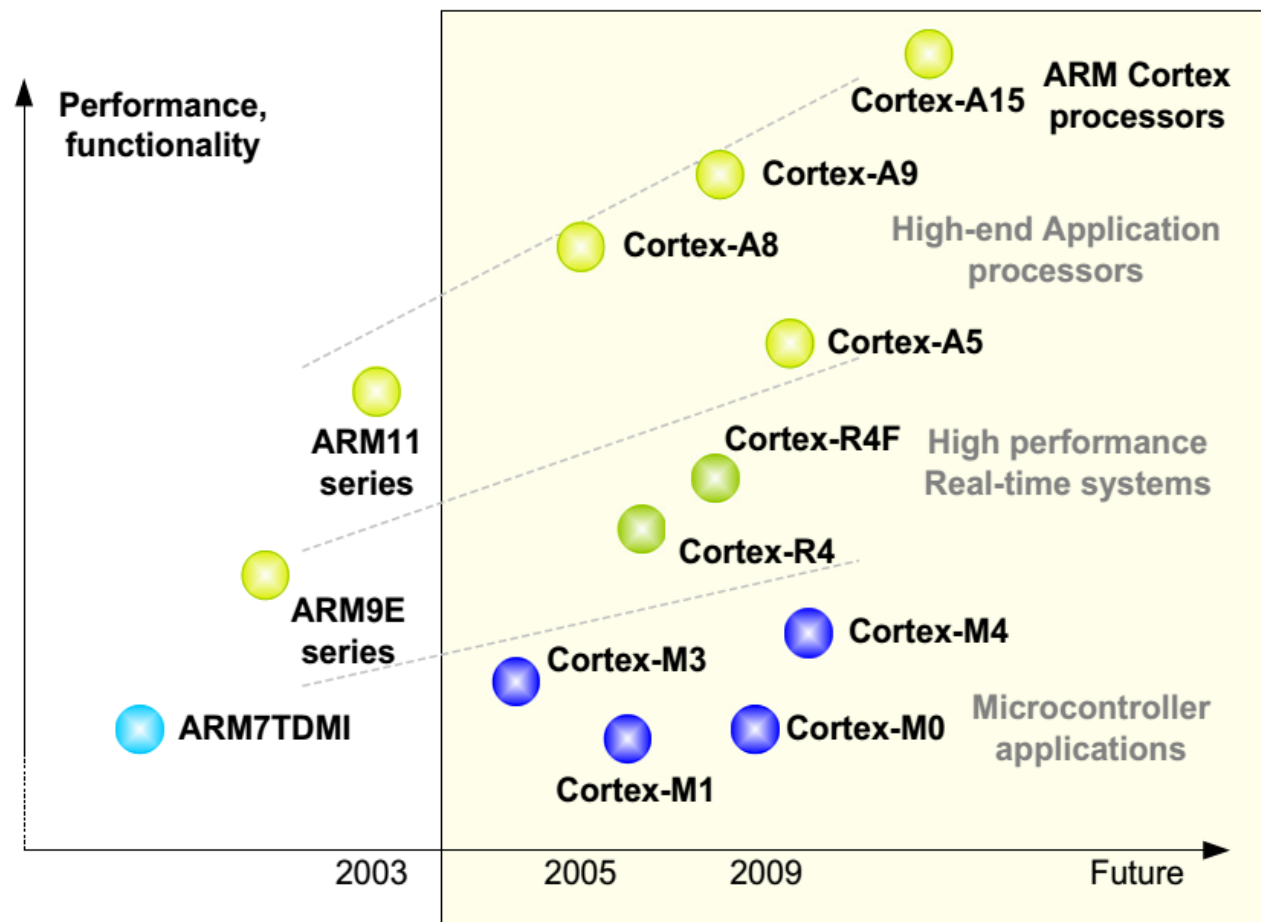
ABOUT ARM

- Founded in 1990 in UK, 員工超過4千人, 營業額約US\$1.66B (NT\$500yee) in 2016。主要產品是RISC CPU的核心。於2016年被日本軟銀買走。
- ARM develops intellectual property (IP) blocks which are used in silicon chips。IP (Intellectual Property智慧財產權) 授權就是把自己所設計創造的東西, 以某種方式授權給其他人利用, 藉以收取授權費。
- Customers combine ARM IP with their own IP to create complete chip designs
- ARM earns license fees when we deliver ARM IP to our partners and royalties when our partners ship chips that contain ARM IP



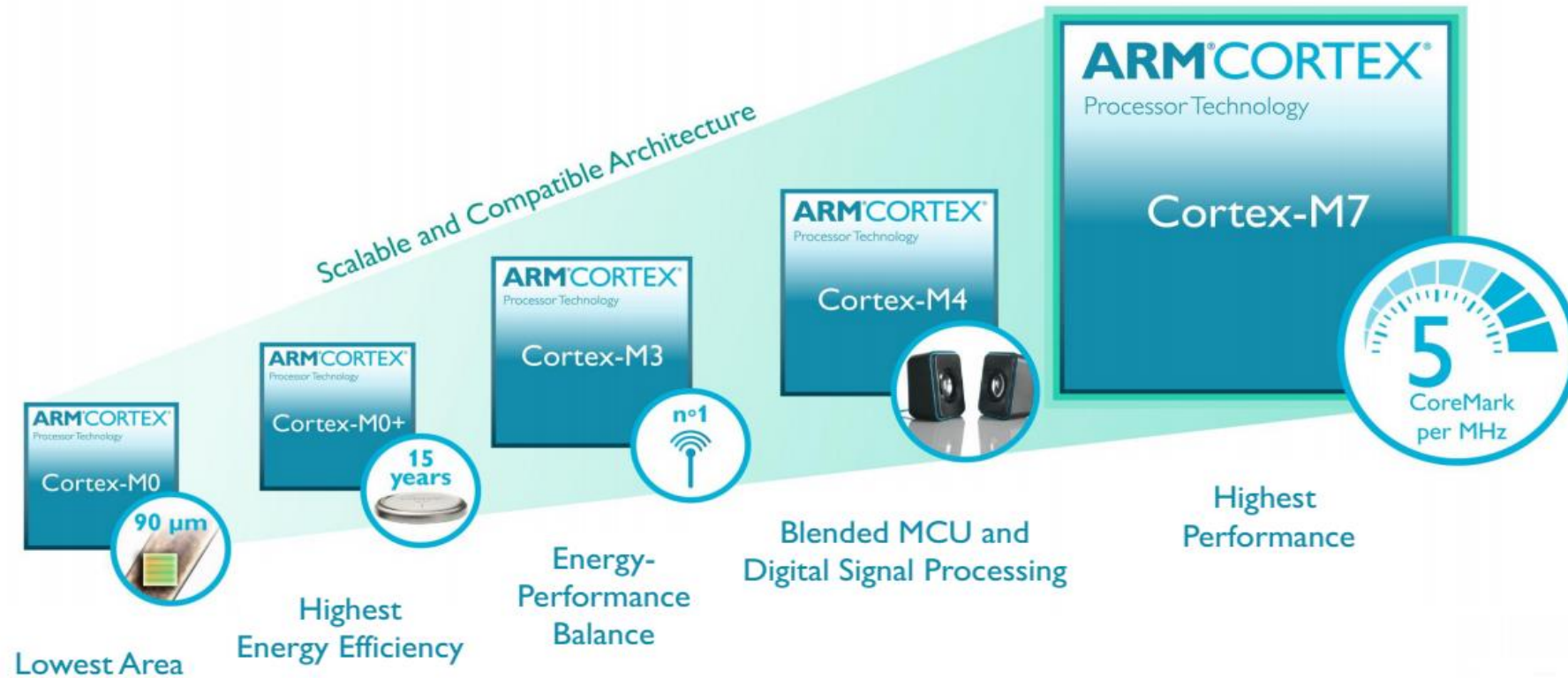
ARM的高中低階CORTEX系列

- ARM在1993年推出的ARM7核心，由於製程的進步，所以ARM7開始大賣。ARM在ARM11之後的處理器家族，改採Cortex命名，並針對高、中、低階分別劃分為A、R、M三大處理器，分別使用不同的處理器架構，ARMv7-A、ARMv7-R、ARMv7-M三種。
- 高階手機用的Cortex-A系列，或者是微控制器所使用的Cortex-M系列，需要即時處理的系統則用Cortex-R系列。

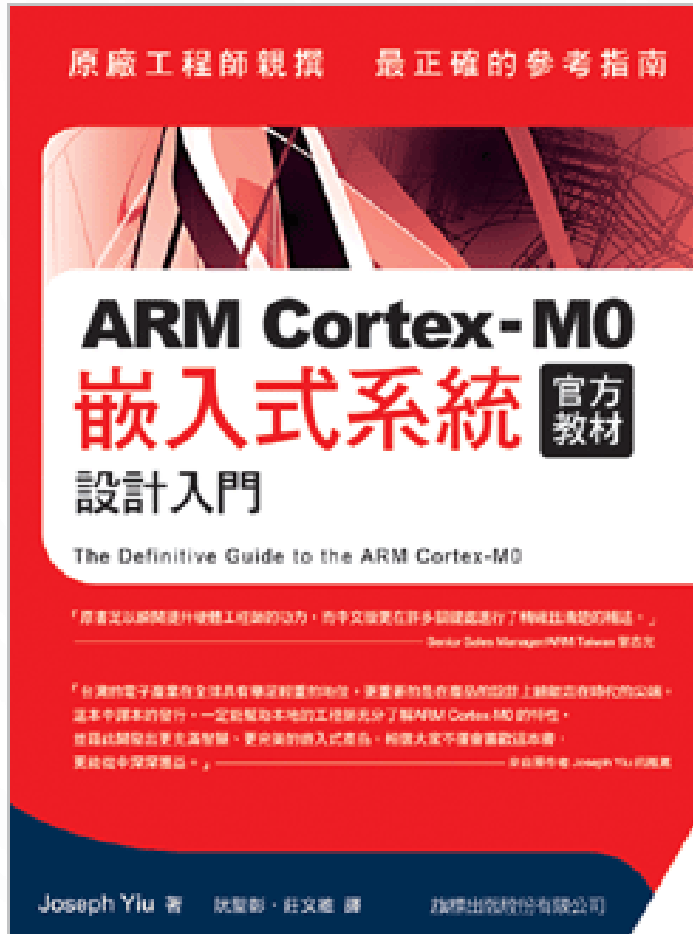


ARM cortex-m series for MCU

- 依照不同市場區隔區分成M0/M0+, M3, M4 至M7，其中，Cortex-M0/M0+用來取代8/16位元應用，Cortex-M3可做更複雜的應用，而Cortex-M4 和M7則可拿來做高階的DSP運算。



ARM cortex-m0 官方教材- 嵌入式系統設計入門



作者：Joseph Yiu著、阮聖彰譯、莊文維譯

書號：F8783

ISBN：9789863122210

定價：780 元

附件：無附光碟片

本書由 ARM 原廠工程師親自撰寫，並經過官方認可的訓練教材。

透過本書您將可以確實了解 Cortex-M0 處理器的軟硬體架構，並同時學習 Keil MDK (Microcontroller Development Kit)、GNU 工具鏈與 ARM RVDS (RealView Development Suite) 的開發環境，正確掌握開發的程序、除錯以及各種注意事項，為進入嵌入式系統的開發做好紮實的準備。

STM nucleo-f072rb



Part # / Keyword



☐ Stocked
☐ RoHS C

Products

Manufacturers

Applications

Services & Tools

Catalog

Help

[All Products](#) > [Embedded Solutions](#) > [Engineering Tools](#) > [Embedded Development Tools](#) > [Embedded Processor Development Kits](#) > [Development Boards & Kits - ARM](#) >

[See an Error?](#)



[Enlarge](#)

Images are for reference only
See Product Specifications

Mouser Part #:

511-NUCLEO-F072RB

Manufacturer Part #:

NUCLEO-F072RB

Manufacturer:

[STMicroelectronics](#)

Description:

Development Boards & Kits - ARM STM32 Nucleo-64
development board with STM32F072RB MCU

[Learn more about STMicroelectronics NUCLEO-F072RB](#)

[NUCLEO-F072RB Datasheet](#)

[Add to Compare List](#)

[Share](#) | [Email](#) [Facebook](#) [Google+](#) [Twitter](#)

[Specifications](#)

[Documents \(1\)](#)

[My Notes](#)

Manufacturer:

STMicroelectronics



Real Time Availability

Stock: 112 Can Ship
Immediately

On Order: 0

Factory Lead-Time: 6 Weeks

Enter Quantity:

[Buy](#)

Minimum: 1
Multiples: 1

Pricing (TWD)

1: NT\$368.56